

AutoResearchClaw でできること — 詳細ガイド

取り込んだリポジトリ `aiming-lab/AutoResearchClaw` (MIT ライセンス) の機能をまとめた日本語解説です。原典: <https://github.com/aiming-lab/AutoResearchClaw> / 論文: arXiv:2605.20025 本リポジトリ内のコピーは元リポジトリと git 連携を断っており、誤って upstream へ push/PR されることはありません。

⚠ 実運用での重要な但し書き (2026-06-14 追記): 本ツールで実際に CIM 研究を 1 本走らせ、生成された論文 **PumpBench** の実験を **本物のモデルで再検証**したところ、生成実験は簡略化した「おもちゃモデル」で動いており、**定量結論の一部は実装で再検証すると成り立たなかった**。詳細と実測は §13。生成された数値結論は、**本物の実装・実インスタンスで再実行して確定すること**。

1. ひとことで言うと

「研究テーマを一言渡すと、本物の文献調査・実験・査読・LaTeX 整形まで全自動でこなし、学会投稿レベルの論文を一本書き上げる AI 研究パイプライン」です。

```
researchclaw run --topic "あなたの研究アイデア" --auto-approve
```

これだけで、テーマの分解 → 文献収集 → 仮説生成 → 実験コード生成・実行 → 結果分析 → 論文執筆 → 査読 → LaTeX 出力 までが一気通貫で走ります。完全自動でも、要所だけ人間が口を出す「副操縦士 (Co-Pilot) モード」でも動かせます。

入力と出力

	内容
入力	研究テーマ (自然文の一文) + LLM の API キー、または ACP 対応エージェント (Claude Code 等)
出力	完成した論文一式 (下表)。 <code>artifacts/rc-<日時>-<hash>/deliverables/</code> にまとめて出力される

生成される成果物

ファイル	中身
<code>paper_draft.md</code>	論文本文 (序論・関連研究・手法・実験・結果・結論、5,000〜6,500 語)

ファイル	中身
paper.tex	学会テンプレート準拠の LaTeX (NeurIPS / ICLR / ICML 切替可、Overleaf でそのままコンパイル可)
references.bib	OpenAlex / Semantic Scholar / arXiv から取得した 実在する BibTeX 文献
verification_report.json	引用の 4 層整合性チェック結果 (捏造引用は自動削除)
experiment_runs/	生成された実験コード + サンドボックス実行結果 + 構造化 JSON メトリクス
charts/	エラーバー・信頼区間つきの条件比較グラフ
reviews.md	マルチエージェント査読レポート (手法と証拠の整合性チェック付き)
evolution/	その run から抽出された「学んだ教訓」
deliverables/	最終成果物を一箇所にまとめたフォルダ

2. 中核となる 23 ステージ・8 フェーズのパイプライン

研究の流れそのものを 23 段階に分解して順番に実行します。途中で失敗すれば自己修復し、仮説が外れれば方向転換 (PIVOT) します。

フェーズ A: 研究スコープ確定		フェーズ E: 実験実行	
1. TOPIC_INIT		12. EXPERIMENT_RUN	
2. PROBLEM_DECOMPOSE		13. ITERATIVE_REFINE	← 自己修復
フェーズ B: 文献探索		フェーズ F: 分析と意思決定	
3. SEARCH_STRATEGY		14. RESULT_ANALYSIS	← 多エージェント分析
4. LITERATURE_COLLECT	← 実API	15. RESEARCH_DECISION	← PIVOT/REFINE 判断
5. LITERATURE_SCREEN	[ゲート]		
6. KNOWLEDGE_EXTRACT		フェーズ G: 論文執筆	
		16. PAPER_OUTLINE	
フェーズ C: 知識統合		17. PAPER_DRAFT	
7. SYNTHESIS		18. PEER_REVIEW	← 証拠チェック
8. HYPOTHESIS_GEN	← 討論	19. PAPER_REVISION	
フェーズ D: 実験設計		フェーズ H: 仕上げ	
9. EXPERIMENT_DESIGN	[ゲート]	20. QUALITY_GATE	[ゲート]
10. CODE_GENERATION		21. KNOWLEDGE_ARCHIVE	
11. RESOURCE_PLANNING		22. EXPORT_PUBLISH	← LaTeX 出力
		23. CITATION_VERIFY	← 関連性チェック

各フェーズの役割

フェーズ	何をするか
A: スコープ確定	テーマを研究課題（リサーチクエスチョン）の木構造に分解。GPU/MPS/CPU を自動検出してコード生成方針を調整
B: 文献探索	OpenAlex → Semantic Scholar → arXiv の順で実在論文を多源検索 → 関連性で選別 → 知識カード化
C: 知識統合	知見をクラスタリングし研究ギャップを特定 → 多エージェント討論で検証可能な仮説を生成
D: 実験設計	実験計画を立て、ハードウェアを考慮した実行可能 Python を生成、必要リソースを見積り
E: 実験実行	サンドボックスで実験を実行。NaN/Inf や実行時バグを検知し、LLM による的を絞った修復で自己回復
F: 分析・判断	結果を多エージェントで分析し、PROCEED（続行） / REFINE（微調整） / PIVOT（方向転換）を自律判断
G: 執筆	章立て → 章ごとに執筆 → 査読（手法と証拠の整合性チェック） → 推敲（分量ガード付き）
H: 仕上げ	品質ゲート → 知識アーカイブ → 学会テンプレで LaTeX 出力 → 引用の整合性・関連性検証

- **ゲートステージ（5, 9, 20）**：人間の承認待ちで一時停止（`--auto-approve` で自動承認、却下するとロールバック）。
- **判断ループ**：ステージ 15 が REFINE（→13 へ戻る）や PIVOT（→8 へ戻る）を起動。成果物は自動でバージョン管理。

3. 際立った特徴（信頼性まわり）

機能	内容
多源の実文献収集	OpenAlex / Semantic Scholar / arXiv から実在論文を取得。クエリ拡張・重複排除・障害時のグレースフル劣化（サーキットブレーカ）
4 層の引用検証	① arXiv ID 照合 → ② CrossRef/DataCite の DOI 照合 → ③ Semantic Scholar タイトル一致 → ④ LLM 関連性スコア。 捏造引用は自動削除
ハードウェア適応実行	NVIDIA CUDA / Apple MPS / CPU-only を自動検出し、コード・import・実験規模を合わせて生成
サンドボックス実験	AST 検証済みコード、不変ハーネス、NaN/Inf 即時失敗、自己修復、最大 10 回の反復改善、部分結果の保存
学会級の執筆	NeurIPS/ICML/ICLR テンプレ、章ごと執筆、捏造防止ガード、分量ガード、免責文の自動排除

機能	内容
捏造防止 (Anti-Fabrication)	VerifiedRegistry が論文に「実際の実験データ」を強制。失敗実験を自動診断・修復してから執筆。未検証の数値はサニタイズ
品質ゲート	人間が介入できる 3 つのゲート (ステージ 5, 9, 20)。ロールバック対応
再現性	全ステージ成果物に SHA256 チェックサム、不変マニフェスト、多段の取り消し (バージョン付きスナップショット)
コスト・ガードレール	予算監視 (50%/80%/100% で警告)。予算超過でパイプライン自動停止

注意 (実測に基づく) : 「捏造防止 (Anti-Fabrication)」が保証するのは *引用と数値の出所が実在すること* であって、*実験モデルが現実を忠実に再現していること* ではない。サンドボックス実験は計算量制約から簡略化モデル (小さい T/N 、近似ダイナミクス) になりがちで、**絶対性能や手法ランキングは本物の実装での再検証が必須**。実例は §13。

4. 人間参加 (HITL) 副操縦士モード

完全自動から一手ずつ確認まで、関与の深さを 7 段階から選べます。

モード	コマンド	挙動
完全自動	<code>--auto-approve</code>	人間の介入なし (従来動作)
ゲートのみ	<code>--mode gate-only</code>	3 つのゲート (5, 9, 20) でのみ承認待ち
チェックポイント	<code>--mode checkpoint</code>	各フェーズ境界 (8 箇所) で停止
副操縦士	<code>--mode co-pilot</code>	重要ステージで深く協働、他は自動
一手ずつ	<code>--mode step-by-step</code>	全ステージ後に停止 (パイプライン学習向け)
エクスプレス	<code>--mode express</code>	最重要 3 ゲートだけ確認
カスタム	<code>--mode custom</code>	<code>stage_policies</code> でステージ別に方針を定義

協働の主要機能

機能	内容
Idea Workshop	仮説をプレスト・評価・洗練 (ステージ 7-8)
Baseline Navigator	AI がベースライン提案 + 人間が追加/削除 + 再現性チェックリスト (ステージ 9)

機能	内容
Paper Co-Writer	章ごとに人間が編集し AI が磨く共同執筆（ステージ 16-19）
SmartPause	確信度に応じて「人間の判断が要りそうな箇所」で自動的に一時停止
Claim Verification	収集文献に照らして AI 生成文をその場で事実確認。根拠のない主張に警告
Intervention Learning (ALHF)	あなたのレビュー傾向を学習し、今後の停止判断を最適化
Branch Exploration	パイプラインを分岐させ複数仮説を並行探索 → 比較 → 最良を統合
エスカレーション	放置時は段階通知（端末 → Slack → メール → 自動停止）

別端末から走行中のパイプラインを操作するコマンドも用意されています。

```
researchclaw attach  artifacts/rc-2026-xxx                # 停止中のパイプラインに接続
researchclaw status  artifacts/rc-2026-xxx                # 状態確認
researchclaw approve artifacts/rc-2026-xxx --message "LGTM"  # 承認
researchclaw reject  artifacts/rc-2026-xxx --reason "ベースライン不足" # 却下
researchclaw guide    artifacts/rc-2026-xxx --stage 9 --message "ResNet-50 を主ベースラインに" # 指示注入
```

5. 分野特化の実験エージェント（v0.5.0～）

実験ステージ（10～13）は、デフォルトの機械学習サンドボックスだけでなく、研究分野に応じて専門エージェントへ自動振り分けされます。

分野	エージェント / 手段
機械学習（既定）	Python サンドボックス（GPU/MPS/CPU 適応）
高エネルギー物理（HEP）	ColliderAgent: Lagrangian → FeynRules → MadGraph5 → Delphes（Magnus クラウド経由）
生物学	COBRAPy によるゲノムスケール代謝モデリング
統計学	シミュレーションスタディ・エージェント
化学・材料など	汎用 Docker 実行系

物理モード（`profile=hep_ph / mode=collider_agent`）では、重いシミュレーションをやり直さずに質量点や解析を追加できる**増分実験**（`--incremental-experiment`）にも対応しています。

6. スキルライブラリ（拡張の仕組み）

研究の質を高める「スキル」を読み込みます。**20 個の組み込みスキル**（科学論文執筆、文献検索、化学、生物 ほか）が同梱され、独自スキルも追加可能です。

分類	スキル例	内容
執筆	scientific-writing	IMRAD 構成、引用整形、報告ガイドライン
分野	chemistry-rdkit	分子解析、SMILES、フィンガープリント、創薬
実験	literature-search	系統的レビュー、PRISMA 手法

```
researchclaw skills list # 読み込み済みスキル一覧
researchclaw skills install /path/to/skill # スキル追加（プロジェクト横断で永続）
researchclaw skills validate ./my-skill # SKILL.md 形式チェック
```

スキルはプロンプトに自動注入されるため、手動で有効化する必要はありません。

7. 自己進化（MetaClaw 連携・任意）

[MetaClaw](#) と連携すると、**run をまたいで学習**します。失敗や警告を「教訓」として捕捉 → 再利用可能なスキルへ変換 → 次回 run の全 23 ステージに注入。同じ失敗を繰り返さなくなります。

公開された A/B 実験では、教訓→スキル変換の有無で次の改善が報告されています。

指標	ベースライン	MetaClaw あり	改善
ステージ再試行率	10.5%	7.9%	-24.8%
REFINE サイクル数	2.0	1.2	-40.0%
総合ロバストネススコア	0.714	0.845	+18.3%

デフォルトは OFF・追加依存なしで、既存テスト 2,699 件はすべて通過します。

8. 動かし方いろいろ

API キー方式でも、Claude Code 等の**エージェントを LLM バックエンドにする方式（ACP）**でも動きます。

方法	やり方
スタンドアロン CLI	<code>researchclaw run --topic "..."</code> (自動) / <code>--mode co-pilot</code> (協働)
Python API	<code>from researchclaw.pipeline import Runner; Runner(config).run()</code>
Claude Code	<code>RESEARCHCLAW_CLAUDE.md</code> を読み込ませて「このテーマで研究して」と言うだけ
各種 AI CLI	<code>RESEARCHCLAW_AGENTS.md</code> を文脈に渡せばエージェントが自動セットアップ
OpenClaw	リポジトリ URL を渡し「Xを研究して」と言うと、clone・install・設定・実行まで自動

ACP 対応バックエンド (API キー不要)

`acpx` 経由で、以下の coding エージェントをそのまま LLM として使えます。23 ステージ全体で単一セッションを維持します。

エージェント	コマンド	提供元
Claude Code	<code>claude</code>	Anthropic
Codex CLI	<code>codex</code>	OpenAI
Copilot CLI	<code>gh</code>	GitHub
Gemini CLI	<code>gemini</code>	Google
OpenCode	<code>opencode</code>	SST
Kimi CLI	<code>kimi</code>	Moonshot

主な CLI サブコマンド

`run` / `init` / `setup` / `validate` / `doctor` / `report` / `serve` / `dashboard` / `wizard` / `project` / `mcp` / `overleaf` / `trends` / `skills` / `calendar` / `attach` / `status` / `approve` / `reject` / `guide`

9. クイックスタート

1. インストール

```
git clone https://github.com/aiming-lab/AutoResearchClaw.git
cd AutoResearchClaw
python3 -m venv .venv && source .venv/bin/activate # Windows は .venv\Scripts\activate
pip install -e .
```

2. セットアップ (OpenCode beast mode の導入、Docker/LaTeX の確認)

```
researchclaw setup
```

3. 設定 (対話式で LLM プロバイダを選択し config.arc.yaml を生成)

```
researchclaw init
```

または手動: `cp config.researchclaw.example.yaml config.arc.yaml`

4. 実行

```
export OPENAI_API_KEY="sk-..."
```

```
researchclaw run --config config.arc.yaml --topic "あなたの研究アイデア" --auto-approve
```

最小構成の設定例

```
project:
  name: "my-research"
research:
  topic: "あなたの研究テーマ"
llm:
  base_url: "https://api.openai.com/v1"
  api_key_env: "OPENAI_API_KEY"
  primary_model: "gpt-4o"
  fallback_models: ["gpt-4o-mini"]
experiment:
  mode: "sandbox"
  sandbox:
    python_path: ".venv/bin/python"
```

実験の実行環境は `simulated` (模擬) / `sandbox` (ローカル隔離) / `docker` / `ssh_remote` (GPU サーバ) から選べます。

10. ベンチマーク (ARC-Bench)

自律研究の評価用に **55 トピックのオープンエンドな研究ベンチマーク** ARC-Bench を同梱 (`experiments/arc_bench/`、Hugging Face でも公開)。

- 内訳: ML 25 / HEP 10 / 量子 10 / 生物 7 / 統計 3
 - 各トピックに「研究課題 + 条件 + 指標 + データセット」のマニフェストと、採点用ループリックが付属
-

11. 必要環境・前提・注意

- **Python 3.11+**。テストは `pytest` で 2,699 件。
 - LLM の API キー（OpenAI 互換 / OpenRouter / DeepSeek / MiniMax 等）**または** ACP 対応エージェント。
 - 実験を本格実行するには **Docker** や **LaTeX**（PDF コンパイル用）、GPU があると望ましい。
 - 文献収集は外部 API（OpenAlex / Semantic Scholar / arXiv）に依存。Semantic Scholar はキー登録でレート上限が緩和。
 - 公式の主動作環境は Linux/macOS 系。Windows で動かす場合はパス・仮想環境の有効化コマンド等に読み替えが必要。
-

12. このリポジトリ内コピーについて（安全性）

- 本コピーは `cim-cac-maxcut-g22` 配下に**通常ファイル**として取り込み済み（`.git` 削除済み・`submodule` 化も解消済み）。
 - 元の `.gitignore` は `gitignore.autoresearchclaw.txt` にリネームし、全ファイルを本リポジトリにコミット済み。
 - **push 先は自分のリポジトリ `hikaru122700/cim-cac-maxcut-g22` のみ**。元リポジトリ（`aiming-lab`）へ誤って PR/push されることは構造的に発生しません。自由に改変できます。
-

13. 実運用での検証所見 — PumpBench を G22 で本物モデル再検証（2026-06-14）

本資料の §1–§12 は README ベースの機能紹介だが、本節は **実際にこのツールで CIM 研究テーマを 1 本走らせ**（テーマ「CIM のポンプ電力関数の最適化」）、生成された論文 **PumpBench** の実験を**本物の実装で再検証**して得た実運用所見である。これがツールの実力と限界を最も正直に表す。

13.1 何が生成されたか

- 論文一式 `PumpBench: Learned CIM Pump Schedules Fall Short of a Tuned Sigmoid`（本文・LaTeX・`references.bib`・図・実験コード `code/`）。
- 中心主張: 「**開ループのポンプ形状は小レバー、閉ループフィードバック（CAC）が大レバー**」。8 条件を計算量整合で比較。
- 出力先: `AutoResearchClaw/artifacts/pump_power_opt/deliverables/`。

13.2 落とし穴 — 実験は「おもちゃモデル」だった

- 生成実験は正規形の平均場 CIM ($dx=(-1+p-x^2)x - \xi \cdot Jx$ 、 x^3 飽和、無次元) を **T=100** ステップ・**K=20** 軌道で積分する簡略版だった。
- 実 G-Set 読込機能を持ちながら、**CPU 予算の都合で G22 (N=2000) をスキップ**していた (`data.load_heldout` の `n<=1000` ガード)。
- そのまま G22 で評価すると絶対カット値が **78–96%** と低く、本リポジトリの検証済み実装 (CIM 99.75% / CAC 99.99%) と比較できない。

13.3 本物モデルでの再検証 (G22, 100 trial)

開ループ = `modules/CIM.py` (Inoue–Yoshida ファイバーループ進行波)、閉ループ = `modules/CAC.py` (Leleu 2021) で、同じ条件比較をやり直した。データ: `results/2026-06-14/pumpbench_real_cim_g22/`。

条件	平均	最良	%既知ベスト
線形利得ランプ (最良開ループ, $g_0 \propto k$)	13280.0	13337	99.84%
CAC (本物・閉ループ)	13278.8	13358	99.99%
線形電力ランプ (現行ベースライン)	13275.3	13326	99.75%
シグモイド	13253.7	13313	99.66%
べき乗 早上げ $p=0.5$	13246.9	13306	99.60%

- **全条件 99.6–99.99%** で過去実測と一致 — おもちゃモデルの歪み (78–96%) は消えた。**精度低下 (リグレッション) は無い**。
- 論文の「シグモイド ≫ 線形」は**本物モデルで非再現** — むしろシグモイドは線形より悪い (指数飽和のファイバー模型では利得の早い立ち上げが不利。リポジトリのポンプ最適化実験 2026-06-08 のシグモイド `z=-22` とも一致)。**具体的なポンプ形状ランキングは転移しない**。
- 一方、**メタ結論 (閉ループ CAC が本丸 / 開ループ形状は小レバー) は本物 G22 でも成立** — CAC のみ BKS-1 (13358) に到達。ただし本物模型では両レバーとも小さい (開ループ形状差 ~0.7%)。

13.4 教訓 — バランスの取れた評価

- **強み:** テーマ分解・文献収集・仮説生成・実験コード骨格・執筆・査読・LaTeX まで一気通貫で「研究の足場」を高速に作る。捏造引用の自動排除も実働した。
- **限界:** サンドボックス実験は計算量制約から**簡略化モデル** (小さい `T / N`、近似ダイナミクス) になりがち。**生成された定量結論・絶対値・手法ランキングは、本物の実装・実インスタンスで再検証するまで信用しない**。「どのモデルで測ったか」を必ず確認する。
- **使い方の指針:** ARC は**仮説生成・構造化・初稿**に使い、**数値結論は検証済み実装で再実行して確定**する。本件 (PumpBench → 本物 G22 再検証) がその実例。詳細考察: `results/2026-06-14/pumpbench_real_cim_g22/RESULTS.pdf`。

出典: *AutoResearchClaw README* (*aiming-lab/AutoResearchClaw*, MIT License)。本資料は同 *README* をもとに日本語で要約・再構成し、§ 13 に本リポジトリでの実運用検証 (2026-06-14) を追記したものである。